Module Code

CS6471

Group Assignment

Artificial Intelligence and Machine Learning for Business

Team Members

Arya Rajan Kolandra
Rince Sebastian
Rohan Franklin Joseph

Group Number

19

Student ID	Student Name	Percentage of Marks
25196197	Arya Rajan Kolandra	32 %
25252984	Rince Sebastian	33 %
25009567	Rohan Franklin Joseph	35 %

Research Question:

How can session-level behavioural patterns be used to classify users into the Browsing, Consideration, or High-Intent stages of the purchase decision process?

Original Dataset: online_shoppers_intention.xlsx

Problem Identification and Initial Dataset

1.1 Problem Definition and Motivation

Understanding user behaviour on e-commerce platforms is essential for improving customer engagement and increasing conversion rates. Traditional models typically treat purchase prediction as a binary classification problem (purchase vs. no purchase), but this oversimplifies the complex behavioural patterns users exhibit while navigating a website.

This project adopts a **three-stage behavioural intention framework**, recognising that visitors transition through distinct cognitive and behavioural phases before making a purchase. Accurately identifying where a visitor lies within this funnel enables more effective personalisation, intervention strategies, and real-time optimisation of user experience.

The three stages considered are:

- **Browsing Stage** — low engagement, exploratory navigation.
- **Consideration Stage** — increased interaction with product-related content.
- **High Purchase Intent Stage** — strong signals of readiness to purchase, including high time investment and revenue-leading behaviour.

The project therefore seeks to analyse behavioural features extracted from web session logs and classify each visitor into one of these three stages, enabling a richer understanding of user journeys beyond simple binary outcomes.

1.2 Research Question and Target Variable

"How can session-level behavioural patterns be used to classify users into the Browsing, Consideration, or High-Intent stages of the purchase decision process?"

The **target variable** created for modelling is `Stage`, a three-class label derived from rules involving:

- Product page frequency
- Product-related duration
- Page value signals
- Actual purchase (`Revenue`)

This multi-stage variable represents the user's inferred position along the intention spectrum.

1.3 Dataset Source and Structure

The dataset used is the **Online Shoppers Purchasing Intention Dataset**, containing **12,330 sessions** and **18 features** spanning behavioural, categorical, and transactional attributes.

Key variable types include:

Behavioural counts and durations:
`Administrative`, `Informational`, `ProductRelated`, and their duration counterparts.

Behavioural rate features:
`BounceRates`, `ExitRates`, `PageValues`.

Categorical attributes:
`Month`, `OperatingSystems`, `VisitorType`, `Region`, `Browser`, `TrafficType`.

Outcome variables:
`Revenue`.

SECTION 1 : Loading Dataset

PART 0

To begin the analysis, we first imported the core libraries required for data handling, visualisation, and later modelling.
This included `pandas` and `numpy` for data manipulation, `matplotlib` and `seaborn` for plotting, and selected components from `scikit-learn` to support preprocessing and model development.

PART 1

After configuring the environment, the raw dataset *online_shoppers_intention.csv* was loaded into a pandas DataFrame using `pd.read_csv()`, providing the initial structure for subsequent exploration and cleaning.

SECTION 2 : Exploratory Data Analysis

PART 2.1 : Basic Structure

To begin the exploratory analysis, we first examined the fundamental structure of the dataset. This included checking the total number of records and features using the DataFrame's shape, followed by an inspection of the data types associated with each column. These steps provide an initial understanding of how the dataset is organised and help identify which variables are numerical, categorical, or boolean.

A preview of the dataset was then displayed using `df.tail()` to verify that the imported data was correctly formatted and contained no structural issues. This initial overview established the foundation for the deeper exploratory analysis performed in subsequent steps.

PART 2.2 : MISSING VALUES & DUPLICATES

We next examined the dataset for any structural issues related to missing or duplicated entries.

The `df.info()` summary was used to confirm that all columns contained 12,330 non-null values.

A column-wise check using `df.isna().sum()` verified that **no missing values** were present in the dataset.

This meant that no imputation or removal steps were necessary for handling null values.

We then checked for duplicate rows using `df.duplicated().sum()`.

The dataset was found to contain **125 duplicate entries**, indicating repeated session records.

Identifying these issues early ensured that data cleaning actions could later address redundancy and maintain dataset integrity.

PART 2.3 : Categorical Variable Exploration

- We converted key categorical columns (`Month`, `VisitorType`, and `Weekend`) to the `category` datatype to ensure they were correctly interpreted during analysis and later one-hot encoding.

- The `Revenue` column was handled separately and converted to an integer format (0 or 1) to standardise the target-related values.

- For each categorical feature, we printed the value counts to understand the distribution of categories within the dataset. Bar plots were generated using `sns.countplot()` to provide a clear visual summary of the frequency of each category.

- These plots highlighted imbalances across several variables, such as the dominance of Returning Visitors and the uneven distribution of sessions across months. We also examined the distribution of the `Weekend` variable, confirming that the majority of sessions took place on weekdays.

- Finally, the distribution of `Revenue` was printed, reaffirming that only a small proportion of sessions resulted in a purchase.

Image reference Cf. 73

Visualising Revenue Counts

We visualised the distribution of the `Revenue` variable to understand how many sessions resulted in a purchase. A bar chart was created using `sns.countplot()`, showing the frequency of `Revenue = 0` (no purchase) and `Revenue = 1` (purchase).

The plot revealed a strong class imbalance, with the majority of sessions falling under the non-purchase category. This imbalance is important because it affects how models learn and makes it necessary to apply techniques like class weighting in later stages.

Understanding this distribution early helped guide decisions made during model design and evaluation.

Image reference Cf. 74

Part 2.4 : Numerical Variable Summary + Shape

We generated a statistical summary of all numerical features using `df.describe().T` to better understand their central tendencies and distributional characteristics. This table provided key metrics such as the mean, standard deviation, minimum, and maximum values for each numeric column.

The summary revealed that several duration-based features, such as `ProductRelated_Duration` and `Informational_Duration`, had very large ranges and high standard deviations, indicating the presence of extreme values and long-tail behaviour.

Page interaction variables like `Administrative`, `Informational`, and `ProductRelated` showed right-skewed distributions, with most users viewing only a small number of pages. Behavioural rate features (`BounceRates` and `ExitRates`) were concentrated near zero, suggesting that a majority of sessions had minimal bounce or exit activity.

The `PageValues` variable displayed a high number of zeros and a long upper tail, which is consistent with typical e-commerce browsing behaviour.

Part 2.5 : DISTRIBUTION ANALYSIS (Histogram)

This summary helped identify features with skewed distributions or potential outliers, supporting later decisions regarding modelling and feature engineering. We identified all numerical columns in the dataset by selecting features with `int64` or `float64` datatypes.

To understand how each numeric variable was distributed, we plotted individual histograms with kernel density estimates (KDE) using `sns.histplot()`.

The `Revenue` column was excluded from this section, as its distribution had already been analysed earlier. Most numeric features displayed **heavily right-skewed distributions**, indicating that a large proportion of users engaged minimally, while a smaller portion showed extremely high interaction levels.

Duration-based variables such as `Administrative_Duration`, `Informational_Duration`, and `ProductRelated_Duration` contained long tails, reflecting occasional sessions with substantial time spent.

Count-based variables like `ProductRelated` and `Informational` also showed steep drop-offs, with the majority of users interacting with only a few pages.

Behavioural rate features (`BounceRates` and `ExitRates`) were concentrated near zero, with a few isolated peaks at higher values. Site attribute variables like `OperatingSystems`,

`Browser`, `Region`, and `TrafficType` revealed uneven category frequencies, consistent with real-world browsing behaviour.

Overall, the histogram analysis highlighted strong skewness, the presence of outliers, and variability in user engagement patterns, guiding later modelling and feature engineering decisions.

Image reference Cf. 76

Part 2.6 : BOX PLOTS FOR OUTLIER DISCOVERY

We grouped similar numeric features to streamline outlier detection.

The groups included:

- Page count features
- Duration features
- Rate features
- Value-based features

These groupings allowed us to analyse related behaviours together in the following boxplots.

Part 2.6(A) Boxplots were generated for the page count features to check for outliers. `Administrative` and `Informational` counts showed low variability. `ProductRelated` had a wider spread with many high-value outliers, indicating varied product engagement across users.

Image reference Cf. 77

Part 2.6(B) We plotted boxplots for the duration-based features (`Administrative_Duration`, `Informational_Duration`, `ProductRelated_Duration`) to examine outliers and variability. All three features showed strong right-skew, with most sessions having very short durations. `ProductRelated_Duration` displayed a significantly wider range and many extreme outliers, indicating that some users spent substantial time on product pages. These patterns highlight large differences in user engagement intensity, which may be meaningful for later stage classification.

Image reference Cf. 78

Part 2.6(C) We plotted boxplots for the behavioural rate features (`BounceRates` and `ExitRates`) to identify outliers and assess variability.
Both variables were heavily concentrated near zero, indicating that most users had low bounce or exit activity.
A noticeable number of outliers appeared at higher rate values, showing that a smaller subset of sessions experienced significantly higher bounce or exit probabilities.
These patterns highlight behavioural differences across users and confirm that rate features contain meaningful variation for modelling.

Image reference Cf. 17

Part 2.6(D) We generated boxplots for the value-based features `PageValues` and `SpecialDay` to inspect their spread and detect outliers. `PageValues` showed a large number of high-value outliers, reflecting that only a small portion of sessions generated substantial page value.
`SpecialDay` remained close to zero for nearly all observations, indicating that very few sessions occurred near a significant shopping-related date. These patterns highlight the sparse and skewed nature of value-related behavioural signals within the dataset.

Image reference Cf. 22

Part 2.7 Correlation Analysis (Numeric Features)

- A correlation matrix was generated for all numeric features to understand linear relationships within the dataset.
- Strong positive correlations appeared within related groups, such as page counts and their corresponding durations (e.g., `ProductRelated` and `ProductRelated_Duration`).
- Bounce and exit rates were also moderately correlated, reflecting similar user navigation behaviour.
- Most other features showed weak correlations, indicating they capture different aspects of user activity.
- This analysis helped identify feature dependencies and informed later modelling decisions.

Image reference Cf. 79

Part 2.8 : Target Relationships (Revenue vs Categorical Predictors)

We examined how the target variable (`Revenue`) varies across key categorical features using bar plots of mean revenue rate.

- **Month:** Certain months, such as May and November, showed noticeably higher purchase rates, indicating seasonal shopping patterns.
- **VisitorType:** New visitors had the highest average revenue rate, while returning visitors showed lower conversion likelihood.
- **Weekend:** Weekend sessions showed slightly higher purchase rates compared to weekdays.

These insights helped identify categorical factors linked to purchase behaviour, informing later feature engineering and model development.

Image reference Cf. 80

SECTION 3 : Initial Data Preparation

PART 3.1 : Importing Libraries

To begin the preparation stage, the required library **pandas** was imported. This allows for structured data manipulation, enabling tasks such as cleaning, transforming, and validating the dataset.

PART 3.2 : Removing Duplicate Rows

The dataset was checked for duplicated entries using `df.duplicated().sum()`, revealing **125 duplicate rows**.
These were removed with `df.drop_duplicates()`, ensuring that repeated sessions do not bias descriptive statistics or model behaviour.

PART 3.3 : Correcting Data Types

Several columns required type correction to ensure consistency:

- **Revenue** and **Weekend** were converted to integers (0/1).
  The code handles values robustly—whether originally stored as boolean, strings (`"True"/"False"`), or numbers.

- **Month** and **VisitorType** were converted to **categorical** types, improving memory efficiency and preparing them for downstream encoding.

The updated `dtypes` confirm that all fields now align with their intended roles in analysis and modelling

PART 3.4 : Missing Values Check

A complete missing-value audit was conducted using `df.isnull().sum()`.
The output confirmed that **no columns contained missing entries**, indicating a fully populated dataset and removing the need for imputation at this stage.

PART 3.5 — Outlier and Range Validation

Summary statistics were generated for all numeric fields using `describe().T`.
This allowed an initial scan for unusual distributions or extreme values.
At this point, the analysis was kept observational—**no transformations or removals were applied**, as the goal was only to understand natural variability within the data.

PART 3.6 — Negative Value Checks

Key duration- and count-related columns were checked to ensure all values were non-negative.
Each variable returned **zero negative entries**, confirming that the dataset aligns with expected behavioural patterns (e.g., page counts and time durations cannot be below zero).

PART 3.7 : Validating Behavioural Rate Columns

BounceRates and ExitRates were validated to ensure values fall within the logical range **0 to 1**.
Both checks confirmed that **all records remained within bounds**, with no invalid rates detected.

PART 3.8 : Creating Total_Duration Feature

A new aggregate feature, **Total_Duration**, was created by summing session time across administrative, informational, and product-related page categories.
This combined metric captures overall user engagement and will serve as a useful behavioural indicator in later modelling stages.

Preview the new feature

After generating the new `Total_Duration` feature, a small preview of the relevant columns was displayed to confirm that the calculation executed correctly.
The output shows that for each row, `Total_Duration` accurately reflects the sum of the three underlying duration components, validating the feature prior to modelling.

Table reference Cf. 84

PART 3.9 : Creating the Purchase Intent Stage (Browsing / Consideration / HighIntent)

To support multi-class modelling of user behaviour, a new categorical target variable called **Stage** was created. This feature groups each session into one of three intent levels based on observable browsing patterns:

- **Browsing** (default): minimal engagement.
- **Consideration**: moderate interaction with product pages.
- **HighIntent**: strong behavioural signals or confirmed purchase.

Custom rule-based masks were developed using product page counts, time spent, PageValues, and the Revenue outcome.
HighIntent was assigned first to capture strong signals, followed by Consideration for mid-level engagement, ensuring no overlap between categories.

The resulting distribution showed that the dataset contains a balanced spread across behavioural stages, with a majority in Consideration, followed by HighIntent and Browsing.
This derived feature forms the foundation for the multi-class classification model used later in the project.

PART 3.10 — Saving the Cleaned Dataset

After completing all preprocessing steps, the cleaned and feature-enhanced dataset was exported as a new CSV file using `df.to_csv()`.
This ensures that all subsequent modelling work is performed on a consistent, processed version of the data, free from duplicates, type inconsistencies, and enriched with derived features such as *Total_Duration* and *Stage*.

```
Cleaned dataset saved as: clean_online_shoppers_intention.xlsx
```

SECTION 4 : Iterative Process

To ensure a rigorous investigation of the research question and identify the most effective approach for classifying shopper intention stages, four different modelling strategies were implemented. Each model served a distinct analytical purpose and addressed specific limitations observed in earlier iterations. The progression reflects standard machine-learning best practices, where model complexity and feature engineering increase gradually to avoid premature overfitting and to ensure interpretability.

PART 4.1 : Imports and Path Setup

- We imported the core libraries needed for modelling: `numpy`, `pandas`, `matplotlib`, `seaborn`, and key classes from `scikit-learn` (train–test split, metrics, Logistic Regression, Decision Tree, Random Forest, and `GridSearchCV`).

- `joblib` was imported to allow saving trained models if required.

- File paths were defined:

    - `DATA_PATH` points to the cleaned dataset (`clean_online_shoppers_intention.csv`).
    - `OUTPUT_DIR` specifies a folder (`model_outputs`) where model artefacts and plots can be stored.

- `os.makedirs(OUTPUT_DIR, exist_ok=True)` ensures this output directory exists before training begins.

PART 4.2 : Loading the Modelling Data and Adding Behaviour Features

- The cleaned dataset produced in Section 3 was reloaded from `DATA_PATH` into a DataFrame.

- A helper function `add_behavior_features()` was defined to guarantee that key engineered features are present before modelling.

- Inside this function:

    - `Total_Duration` is created as the sum of the three duration variables if it does not already exist.

    - `Engagement_Score` combines `PageValues` with time spent (converted to minutes) to capture overall engagement.
    - `Navigation_Intensity` is defined as the total number of pages visited across administrative, informational, and product-related categories.

- The function is applied to the cleaned data to create `df_model`, and the shape and column names are printed to confirm that all behavioural features are available for the subsequent modelling steps.

PART 4.3 : Preparing Base Modelling Data

A reusable helper function, **`get_base_data()`**, was defined to standardize how the dataset is prepared before training each model. This ensures consistency across all model iterations. The function performs four key tasks:

1. Target Extraction

- The function checks that the **Stage** variable exists (created earlier in Part 3).
- `Stage` is assigned as the target **y**, representing the three-class intent labels.

2. Preventing Data Leakage

- Columns that could leak final purchase outcomes into the model were removed:
    - `Stage` (target)
    - `Revenue` (boolean purchase indicator)
- Removing `Revenue` ensures that models learn intent patterns rather than directly reading the final purchase outcome.

3. One-Hot Encoding Categorical Variables

- All object, categorical, and boolean columns were automatically identified.
- These variables were converted into numerical form using **one-hot encoding** with `drop_first=True` to avoid multicollinearity.

4. Train–Test Split with Stratification

- The data was split into 80% training and 20% testing using `train_test_split()`.

- **Stratification** was applied to maintain the same distribution of Browsing, Consideration, and HighIntent classes in both sets.

- The function prints the shapes and class distributions of the splits for verification.

Finally, the function returns:
`X_train, X_test, y_train, y_test` — the standardized input and target sets used across all modelling iterations.

A custom evaluation function, **`evaluate_model()`**, was created to standardise how each model's performance is assessed throughout the iterative process. This ensures that all classifiers are compared using identical metrics and visualisations.

Key Components of the Function

1. Classification Report

- The function prints a labelled header for clarity.
- `classification_report()` is used to generate precision, recall, F1-score, and support values for all three intent classes:

    - *Browsing*
    - *Consideration*
    - *HighIntent*

This provides a detailed view of how well a model performs across each behavioural category.

2. Confusion Matrix Construction

- A confusion matrix is computed using `confusion_matrix()`, with class labels sorted to maintain consistent ordering.
- This matrix shows where the model predicts correctly and where misclassifications occur between the three intent stages.

3. Visualisation

- The matrix is displayed using `ConfusionMatrixDisplay`, plotted with a blue colour map for interpretability.
- A standardized figure size and layout are applied to keep all evaluation visuals consistent across iterations.

PART 4.5 : Iteration 0: Baseline Dummy Classifier

To establish a reference point for model performance, a **DummyClassifier** was used as the baseline model.
This classifier predicts only the **most frequent class** (Consideration) regardless of input features.

Image reference Cf. 15

Why this step is important

- It provides a minimum benchmark that all real models must outperform.
- It helps detect whether later models are genuinely learning patterns or only replicating class imbalances.
- It highlights the difficulty of the task: even with no learning, accuracy may appear deceptively high due to class imbalance.

What the code does

- The modelling data is prepared using `get_base_data()`.
- A DummyClassifier with `strategy="most_frequent"` is trained.
- Predictions are generated and evaluated with `evaluate_model()`.

Result Summary

- The model predicts only the Consideration class for all samples.
- **Browsing** and **HighIntent** receive **zero correct predictions**, leading to an F1-score of 0.
- The confusion matrix confirms complete collapse to a single-class prediction.

This baseline now serves as the **starting point** for evaluating more sophisticated models in upcoming iterations.

PART 4.6 : Iteration 1: Multinomial Logistic Regression (Baseline)

After establishing the DummyClassifier baseline, the first real predictive model was trained using **Multinomial Logistic Regression**, a strong starting point for multi-class classification.

Image reference Cf. 126

Why Logistic Regression?

- It provides a **simple but powerful** linear model.
- It establishes a **true learning baseline** to compare against tree-based models later.
- It handles **multiclass problems** naturally through the multinomial formulation.
- It benefits from feature scaling, allowing smoother optimisation.

What the code does

- The training and test data were prepared using `get_base_data()`.
- A modelling pipeline was created with:
  - `StandardScaler()` to normalise feature ranges.
  - `LogisticRegression(multi_class="multinomial")` as the classifier.

- The model was trained, predictions were generated, and performance was evaluated through:
  - a classification report
  - a confusion matrix visualisation

Result Summary

- Logistic Regression achieved a **macro-average F1-score of ~0.89** and an overall accuracy of **~0.90**, a massive improvement over the Dummy baseline.
- All three classes, Browsing, Consideration, and HighIntent were predicted with high precision and recall.
- The confusion matrix shows balanced performance with relatively few misclassifications, particularly between *Browsing* and *Consideration*.

This iteration provides a **strong foundation**, demonstrating that even a linear model can learn meaningful intent patterns from behavioural features.

PART 4.7 : Iteration 2: Decision Tree (class_weight = 'balanced')

After establishing a strong linear baseline with Logistic Regression, a **Decision Tree Classifier** was trained to explore a non-linear modelling approach. Decision trees can naturally capture complex interactions and threshold-based behaviours that are common in user browsing data.

Image reference Cf. 127

Why this step?

- Decision trees **do not require scaling**, making them useful for mixed-type behavioural data.
- They model **non-linear relationships**, which may better capture behavioural differences between Browsing, Consideration, and HighIntent users.
- Setting **class_weight='balanced'** ensures the model compensates for class imbalance internally, without requiring external resampling like SMOTE.

What the code does

- The modelling dataset is re-split using `get_base_data()`, and the class distribution is printed.
- A `DecisionTreeClassifier` is created with:
    - `class_weight='balanced'` to correct for imbalance
    - No depth or leaf restrictions, allowing the model full flexibility

- The model is trained and evaluated using the shared evaluation function.

Result Summary

- The Decision Tree achieved a **macro F1-score of ~0.93**, outperforming Logistic Regression.
- **Browsing** and **Consideration** were classified with very high precision and recall.
- **HighIntent** also performed strongly, though with slightly lower precision due to overlap between Consideration and HighIntent behaviours.
- The confusion matrix shows clear improvement, with fewer cross-class misclassifications.

This iteration confirms that non-linear models can learn richer behavioural structures and improve predictions across all three intent stages.

PART 4.8 — Iteration 3: Random Forest (class_weight = 'balanced')

A **Random Forest Classifier** was introduced to build on the strengths of the earlier Decision Tree model. Random Forests reduce overfitting, improve generalisation, and provide feature importance insights—all valuable for behavioural modelling.

Image reference Cf. 127

Why Random Forest?

- It is an **ensemble** of many trees, giving more stable and robust predictions.
- It captures **non-linear patterns** and interactions between behavioural features.
- Using **class_weight='balanced'** ensures fairness across Browsing, Consideration, and HighIntent despite dataset imbalance.
- It provides **feature importance rankings**, helping identify the strongest predictors of intent.

What the code does

- The dataset is split again using `get_base_data()`, and the class proportions are printed.
- A Random Forest with 200 trees is trained using all features.
- Predictions are evaluated using the shared evaluation function.
- The top 15 feature importances are printed to understand which behaviours drive model decisions.

Result Summary

- Random Forest achieved a **macro F1-score of ~0.96**, outperforming both Logistic Regression and the single Decision Tree.
- All three stages were predicted with high precision and recall.
- The confusion matrix reflects strong separation between classes.
- Top predictors included:
    - **PageValues**
    - **ProductRelated_Duration**
    - **Engagement_Score**
    - **ProductRelated**
    - **Navigation_Intensity**
    - **Total_Duration**

These features align closely with real user engagement patterns, confirming that the model is learning meaningful behavioural signals.

PART 4.9 : Iteration 4: Random Forest + Feature Engineering (Balanced)

In this iteration, the Random Forest model was retrained using an enhanced version of the dataset that included additional **behavioural features** created during earlier preprocessing. The goal was to test whether engineered features improve the model's ability to distinguish Browsing, Consideration, and HighIntent behaviours.

Image reference Cf. 128

Why this step?

- Feature engineering often provides **greater performance gains** than tuning algorithms alone.
- Metrics such as **Engagement_Score**, **Navigation_Intensity**, and **Total_Duration** capture deeper behavioural signals that raw counts cannot fully express.
- Running the same Random Forest with engineered features isolates the effect of these additional insights.

What the code does

- `add_behavior_features()` is applied to ensure engineered metrics exist in `df_fe`.
- The modelling dataset is prepared with `get_base_data()`.
- A Random Forest with class balancing is trained on this expanded feature space.
- Model performance is evaluated using the standard evaluation pipeline.
- The top 15 feature importances are printed to analyse which engineered metrics contribute most.

Result Summary

- The macro F1-score remained very strong at **~0.96**, essentially matching (and slightly refining) the performance from Iteration 3.
- The confusion matrix shows extremely stable performance across all classes, with only small misclassification counts.
- Engineered behavioural features appeared prominently among the top predictors:
    - **PageValues**
    - **ProductRelated_Duration**
    - **Engagement_Score**
    - **Navigation_Intensity**
    - **Total_Duration**

This confirms that engineered behavioural metrics add meaningful predictive value and are essential to modelling user purchase intent accurately.

SECTION 5 — Final Model & Evaluation

PART 5.1 : Hyperparameter Tuning with Grid Search

The final stage of the modelling pipeline involved tuning the Random Forest classifier to extract the best possible performance from the algorithm.
A **GridSearchCV** process was applied to systematically explore combinations of key hyperparameters.

Image reference Cf. 129

Why this step?

- Although previous Random Forest models performed strongly, they used default parameters.
- Tuning allows the model to:
    - Improve generalisation
    - Reduce unnecessary complexity
    - Discover optimal depth, number of trees, and split behaviour
- Using **macro F1-score** as the scoring metric ensures equal importance is given to Browsing, Consideration, and HighIntent classes, regardless of class imbalance.

 # What the code does

- The modelling data is prepared once more using `get_base_data()`.
- A parameter grid is defined to tune four key settings:
    - `n_estimators`
    - `max_depth`
    - `min_samples_split`
    - `min_samples_leaf`
- A base Random Forest with class balancing is wrapped in **GridSearchCV**, using:
    - 3-fold cross-validation
    - F1-macro scoring
    - Parallel processing via `n_jobs=-1`
- The grid search fits 24 candidate models (72 fits across folds).

- The best performing hyperparameters are printed, and the final model is evaluated.

 # Final Model Performance

Using the optimal hyperparameters found during tuning, the final Random Forest model was retrained and tested.

Result Summary

- The tuned model produced **excellent performance**, maintaining the strong accuracy and F1-scores observed in earlier Random Forest iterations.
- All three intent stages were predicted with high precision and recall.
- The confusion matrix confirms stable separation across classes with minimal misclassification.
- This tuned Random Forest therefore represents the **best-performing model** in the pipeline and serves as the final recommended solution for multi-stage purchase intent prediction.

Feature importances for the final Random Forest

To understand which behavioural signals contributed most to predicting user purchase intent, feature importance values were extracted from the final tuned Random Forest model.
The top 15 strongest predictors were visualised in a bar chart.

Image reference Cf. 130

Why this step?

- Helps interpret **which user actions drive intent classification**.
- Validates that engineered features (e.g., Engagement_Score) meaningfully improved the model.
- Provides actionable insights for marketing or UX teams (e.g., which behaviours signal high intent).

Key Insights from the Feature Importance Plot

The most influential predictors were:

1. **PageValues** — the strongest driver of purchase intent, reflecting session value estimation.
2. **ProductRelated_Duration** — time spent on product pages, a key indicator of interest.
3. **Engagement_Score** — combined metric showing overall depth of interaction.
4. **ProductRelated** — number of product pages viewed.
5. **Navigation_Intensity** — total page interactions.
6. **Total_Duration** — overall time on site.

Lower but still meaningful contributors included:

- **ExitRates** and **BounceRates**
- Administrative and informational page behaviours
- Certain categorical dummy variables (e.g., *Month_Nov*, *Region*, *TrafficType*)

Overall Interpretation

The importance ranking confirms that **behaviour rooted in product exploration and session value** is most predictive of HighIntent classifications.
Engineered features such as *Engagement_Score* and *Navigation_Intensity* ranked highly, demonstrating that the feature engineering step added real value to the model's learning process.